

1. Class

Definition:

A **class** is a blueprint or template for creating objects. It defines **fields (variables)**, **methods (functions)**, and **constructors**.

Example:

```
public class Car
{
    public string Brand;    // Field
    public int Year;        // Field

    public void DisplayInfo()    // Method
    {
        Console.WriteLine("Brand : " + Brand + ", Year:" + Year);
    }
}
```

◆ 2. Object

Definition:

An **object** is an instance of a class (real-world entity created from a class).

Object stores the real data like what should be performed

Example:

```
public class Car
{
    public string Brand;

    public int Year;

    public void DisplayInfo()
    {
        Console.WriteLine($"Brand: {Brand}, Year: {Year}");
    }
}
```



```
class Program
```

```

{
    static void Main()
    {
        // Object 1

        Car myCar = new Car();

        myCar.Brand = "Toyota";

        myCar.Year = 2023;

        myCar.DisplayInfo();

        // Object 2

        Car anotherCar = new Car();

        anotherCar.Brand = "Honda";

        anotherCar.Year = 2022;

        anotherCar.DisplayInfo();

    }
}

```

Output:

Brand: Toyota, Year: 2023

◆ 3. Members and Methods

- **Members** → variables, constants, readonly fields, properties.
- **Methods** → functions inside a class that perform actions.

Example:

```

public class Student
{
    public int Id;           // Member
    public string Name;     // Member

    public void ShowDetails() // Method
    {
        Console.WriteLine($"ID: {Id}, Name: {Name}");
    }
}

```

```
Syntax for method
Access modifier Type methodname(parameters)
{
    Method body;
}

public int AddNumbers(int a, int b)
{
    return a + b;
}
```

◆ 4. Access Modifiers

Definition:

They control how accessible class members are.

Modifier	Meaning
public	Accessible everywhere
private	Accessible only inside the same class
protected	Accessible inside the class and derived classes
internal	Accessible within the same project

Example:

```
public class Employee
{
    private int salary;    // only inside Employee
    public string Name;    // accessible everywhere
}
```

◆ 5. Constructors

Definition:

A **constructor** is a special method that initializes objects. It has the same name as the class and no return type.

```
public class Person
{
    public string Name;
    public int Age;

    // Constructor
    public Person(string name, int age)
    {
        Name = name;
        Age = age;
    }
}
```

```

}
class Program
{
    static void Main()
    {
        // When we create an object, constructor is called automatically
        Person p1 = new Person("John", 25);
        Console.WriteLine("Name: " + p1.Name + ", Age: " + p1.Age);
    }
}

```

Types of Constructors:

1. **Default Constructor** – No parameters

```

public class Car
{
    public string Brand;

    // Default constructor
    public Car()
    {
        Brand = "Unknown"; // Default value
    }
}

```

2. **Parameterized Constructor** – With parameters

```

public class Car
{
    public string Brand;

    // Parameterized constructor
    public Car(string brand)
    {
        Brand = brand;
    }
}

```

3. **Copy Constructor** – Copies another object

```

public class Car
{
    public string Brand;

    // Parameterized constructor
    public Car(string brand)
    {
        Brand = brand;
    }

    // Copy constructor

```

```

    public Car(Car c)
    {
        Brand = c.Brand;
    }
}
Car c3 = new Car("BMW");
Car c4 = new Car(c3); // Copy from c3
Console.WriteLine(c4.Brand); // Output: BMW

```

4. **Static Constructor** – Runs once for all objects

Used to initialize **static members** of a class.
 Called **automatically only once** before any object is created.
 Does not take parameters and cannot be called manually.

```

public class Car
{
    public static int Wheels;

    // Static constructor
    static Car()
    {
        Wheels = 4;
        Console.WriteLine("Static constructor called");
    }
}

Car c5 = new Car();
Console.WriteLine(Car.Wheels); // Output: 4

```

5. **Private Constructor** – Restricts object creation

A constructor declared **private** so it **cannot be used to create objects directly**.

Example:

```

public class Person
{
    public string Name;
    public int Age;

    // Default Constructor
}

```

```

public Person()
{
    Name = "Unknown";
    Age = 0;
}

// Parameterized Constructor
public Person(string name, int age)
{
    Name = name;
    Age = age;
}

// Copy Constructor
public Person(Person other)
{
    Name = other.Name;
    Age = other.Age;
}
}

```

Usage:

```

Person p1 = new Person();           // Default
Person p2 = new Person("Ankith", 22); // Parameterized
Person p3 = new Person(p2);         // Copy

```

◆ 6. Constant Members

Definition:

`const` → a fixed value known at **compile time**.

Example:

```

public class MathConstants
{
    public const double Pi = 3.14159;
}

```

Usage:

```

Console.WriteLine(MathConstants.Pi);

```

◆ 7. Readonly Members

Definition:

`readonly` → value assigned at **runtime** (in constructor).

Example:

```

public class Test
{

```

```
    public readonly string CreatedOn;

    public Test()
    {
        CreatedOn = DateTime.Now.ToString();
    }
}
```

Usage:

```
Test t1 = new Test();
Console.WriteLine(t1.CreatedOn);
```

✓ Final Recap

- **Class** → Blueprint (Car design).
- **Object** → Instance (Actual car).
- **Members** → Variables, constants, readonly fields.
- **Methods** → Functions inside a class.
- **Access Modifiers** → Control visibility (public, private, etc.).
- **Constructors** → Initialize objects (default, parameterized, copy, static, private).
- **const** → Fixed at compile time.
- **readonly** → Fixed at runtime.

Inheritance

Inheritance is the mechanism in C# where one class can use the properties and methods of another class.

Defining a Subclass

A subclass (or derived class) is a class that inherits the members (fields, methods, properties) of another class.

The class it inherits from is called the base class (or parent class).

In C#, inheritance is defined using the : symbol.

```
class BaseClass
{
    // members (fields, methods, properties)
}

class SubClass : BaseClass
{
    // new members or overridden members
}
```

```
class Parent
{
    public void Display()
    {
        Console.WriteLine("I am Parent class");
    }
}

class Child : Parent // Child inherits from Parent
{
    public void Show()
    {
        Console.WriteLine("I am Child class");
    }
}

Child c = new Child();
c.Display(); // Inherited
c.Show();   // Own method
```


Single Inheritance

One class derives from one base class.

```
class Vehicle
{
    public void Start()
    {
        Console.WriteLine("Vehicle started.");
    }
}

class Car : Vehicle // Single Inheritance
{
    public void Drive()
    {
        Console.WriteLine("Car is driving.");
    }
}

class Program
{
    static void Main()
    {
        Car c = new Car();
        c.Start(); // From Vehicle
        c.Drive(); // From Car
    }
}
```

2. Multilevel Inheritance

A class is derived from another derived class.

```
class LivingBeing
{
    public void Breathe()
    {
        Console.WriteLine("All living beings breathe.");
    }
}

class Animal : LivingBeing
{
    public void Eat()
    {
        Console.WriteLine("Animals eat food.");
    }
}

class Dog : Animal // Multilevel Inheritance
{
    public void Bark()
    {
        Console.WriteLine("Dog barks.");
    }
}

class Program
{
    static void Main()
    {
        Dog d = new Dog();
        d.Breathe(); // From LivingBeing
        d.Eat();     // From Animal
        d.Bark();    // From Dog
    }
}
```

3. Hierarchical Inheritance

Multiple classes share the same base class.

```
class Shape
{
    public void Draw()
    {
        Console.WriteLine("Drawing a shape.");
    }
}

class Circle : Shape
{
    public void Area()
    {
        Console.WriteLine("Area of circle ");
    }
}

class Rectangle : Shape
{
    public void Area()
    {
        Console.WriteLine("Area of rectangle ");
    }
}

class Program
{
    static void Main()
    {
        Circle c = new Circle();
        c.Draw();
        c.Area();

        Rectangle r = new Rectangle();
        r.Draw();
        r.Area();
    }
}
```

4. Multiple Inheritance (✗ Not allowed with classes in C#)

- A class inherits from **multiple parent classes**.
- **C# does NOT support multiple inheritance of classes** (to avoid the “Diamond Problem”).
- ✓ Instead, **C# supports multiple inheritance through interfaces**.

Visibility Control (Access Modifiers)

- Controls who can access members (fields, methods).
- Common modifiers:
 - `public` → accessible everywhere
 - `private` → accessible only inside same class
 - `protected` → accessible inside class + derived classes
 - `internal` → accessible inside same project

```
class Parent
{
    private int a = 10;
    protected int b = 20;
    internal int c = 30;
    public int d = 40;
}

class Child : Parent
{
    public void Show()
    {
        // Console.WriteLine(a); // ✗ Not accessible (private)
        Console.WriteLine(b); // ✓ Accessible (protected)
        Console.WriteLine(c); // ✓ Accessible (internal and same assembly)
        Console.WriteLine(d); // ✓ Accessible (public)
    }
}
```

Subclass Constructors

- A subclass can call its **base class constructor** using `base()`.

Multilevel Inheritance

- A class inherits from another derived class.

```
class A
{
    public void MethodA()
    {
        Console.WriteLine("Class A");
    }
}

class B : A
{
    public void MethodB()
    {
        Console.WriteLine("Class B");
    }
}

class C : B
{
    public void MethodC() { Console.WriteLine("Class C"); }
}
```

METHOD OVERRIDE

Method overriding is a feature of object-oriented programming where a subclass (child class) provides its own implementation of a method that is already defined in its superclass (parent class), using the same method name, parameters, and return type. The parent method must be marked as `virtual` (or `abstract`), and the child method must be marked as `override` to achieve this.

```
class Parent
{
    public virtual void MethodName()
    {
        // Parent class code
    }
}

class Child : Parent
{
    public override void MethodName()
    {
        // Child class code
    }
}
```

Abstract class

An **abstract class** in C# is a class declared with the keyword `abstract` that **cannot be instantiated directly**.

It is used as a **base class** and may contain:

- **Abstract methods** (methods without body → must be implemented in child classes).
- **Non-abstract methods** (methods with body → can be inherited as-is).

abstract class ClassName

```
{  
    // Abstract method (no body)  
    public abstract void MethodName();  
  
    // Normal method (with body)  
    public void NormalMethod()  
    {  
        Console.WriteLine("This is a normal method inside abstract class");  
    }  
}
```

class ChildClass : ClassName

```
{  
    // Must override abstract method  
    public override void MethodName()  
    {  
        Console.WriteLine("Implementation of abstract method in child class");  
    }  
}
```

Operation Polymorphism

Operation polymorphism means performing a single operation in different ways depending on the type of object that invokes it.

```
class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }

    public double Add(double a, double b)
    {
        return a + b;
    }
}

class Program
{
    static void Main()
    {
        Calculator c = new Calculator();
        Console.WriteLine(c.Add(2, 3));
        Console.WriteLine(c.Add(2.5, 3.5));
    }
}
```


Interface

An **interface** defines a set of methods without implementation.

A class that implements an interface must provide the implementation for all its methods.

```
using System;
```

```
interface IPrintable
{
    void Print();
}
```

```
interface IScannable
{
    void Scan();
}
```

```
class PrinterScanner : IPrintable, IScannable
{
    public void Print()
    {
        Console.WriteLine("Printing document...");
    }

    public void Scan()
    {
        Console.WriteLine("Scanning document...");
    }
}
```

```
class Program
{
    static void Main()
    {
        PrinterScanner device = new PrinterScanner();
        device.Print();
        device.Scan();
    }
}
```

Extending interface

using System;

// Parent interface

interface IVehicle

```
{  
    void Start();  
}
```

// Child interface extending IVehicle

interface IColorVehicle : IVehicle

```
{  
    void Paint(string color);  
}
```

// Class implementing child interface

class Car : IColorVehicle

```
{  
    public void Start()  
    {  
        Console.WriteLine("Car started");  
    }  
  
    public void Paint(string color)  
    {  
        Console.WriteLine("Car painted " + color);  
    }  
}
```

class Program

```
{  
    static void Main()  
    {  
        IColorVehicle myCar = new Car();  
        myCar.Start();      // Output: Car started  
        myCar.Paint("Red"); // Output: Car painted Red  
    }  
}
```

Delegates

A delegate in C# is a type-safe object that holds a reference to a method (or multiple methods).

It works like a function pointer in C/C++, but it is safe and secure because it checks method signatures at compile-time.

Syntax

```
[access_modifier] delegate [return_type] [delegate_name]([parameter_list]);
```

```
public delegate void MyDelegate(string message);
```

```
using System;
```

```
// Declare delegate
```

```
public delegate void GreetingDelegate(string name);
```

```
class Program
```

```
{
```

```
    public static void SayHello(string name)
```

```
    {
```

```
        Console.WriteLine("Hello, " + name);
```

```
    }
```

```
    public static void SayGoodbye(string name)
```

```
    {
```

```
        Console.WriteLine("Goodbye, " + name);
```

```
    }
```

```
    static void Main()
```

```
    {
```

```
        // Assign method to delegate
```

```
        GreetingDelegate greet = SayHello;
```

```
        greet("Ankith"); // Output: Hello, Ankith
```

```
        // Change method dynamically
```

```
        greet = SayGoodbye;
```

```
        greet("Ankith"); // Output: Goodbye, Ankith
```

```
    }
```

```
}
```

Managing errors and exception

Types of errors

1. Compile time error

2. Run time error

Managing errors and exceptions means **handling runtime problems** in your program so that it **doesn't crash**.

- In C#, this is done using **try-catch-finally** blocks.
- You can **catch exceptions** and take action, like showing a message or logging the error.

```
try
{
    // Code that may cause an exception
}
catch (ExceptionType ex)
{
    // Code to handle the exception
}
```

```
using System;

class Program
{
    static void Main()
    {
        try
        {
            int a = 10, b = 0;
            int result = a / b;
            Console.WriteLine("Result: " + result);
        }
        catch (DivideByZeroException)
        {
            Console.WriteLine("Error: Cannot divide by zero!");
        }
    }
}
```

Multiple Catch Statements

```
using System;
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        try
```

```
        {
```

```
            Console.WriteLine("Enter a number:");
```

```
            int num = Convert.ToInt32(Console.ReadLine());
```

```
            int result = 10 / num;
```

```
            Console.WriteLine("Result = " + result);
```

```
        }
```

```
        catch (DivideByZeroException)
```

```
        {
```

```
            Console.WriteLine("Error: You cannot divide by zero!");
```

```
        }
```

```
        catch (FormatException)
```

```
        {
```

```
            Console.WriteLine("Error: Please enter numbers only!");
```

```
        }
```

```
        catch (Exception ex)
```

```
        {
```

```
            Console.WriteLine("Other error: " + ex.Message);
```

```
        }
```

```
    }
```

```
}
```

```
// MessageBox.Show("Please enter valid input. " + ex.Message);
```

General Catch Handler

- A **general catch handler** catches **any type of exception**, regardless of its type.

Useful when you don't know exactly what exception might occur

```
using System;
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        try
```

```
        {
```

```
            Console.Write("Enter a number: ");
```

```
            int num = Convert.ToInt32(Console.ReadLine()); // ✗ FormatException if text
```

```
            int result = 10 / num; // ✗ DivideByZeroException if 0 entered
```

```
            Console.WriteLine("Result = " + result);
```

```
        }
```

```
        catch (Exception ex) // General catch
```

```
        {
```

```
            Console.WriteLine("An error occurred: " + ex.Message);
```

```
        }
```

```
        Console.WriteLine("Program continues...");
```

```
    }
```

```
}
```

Using Finally statement

In **exception handling**, the `finally` block is used to execute code **no matter what happens** — whether an exception occurs or not.

```
try
{
    // Code that might cause an exception
}
catch (Exception ex)
{
    // Code to handle the exception
}
finally
{
    // Code that will always run
}
```

Example

```
try
{
    int x = 10;
    int y = 0;
    int z = x / y; // Will throw DivideByZeroException
}
catch (DivideByZeroException ex)
{
    Console.WriteLine("Cannot divide by zero.");
}
finally
{
    Console.WriteLine("This will always execute.");
}
```